

PHASE 1 REPORT: PROBLEM DEFINITION & SYSTEM DESIGN

Course: Compiler Design Laboratory | Review 1 Submission

Student Name: Lakshya Singh | Reg No.: 24BDS0054

Project Title: Sudarshan: An End-to-End Compiler and Virtual Machine

Repository: https://github.com/laksh1357/MyOWN_Compiler

1. Abstract & Problem Statement

Sudarshan is an individual Compiler Design Laboratory project implementing a modular 5-phase compiler and virtual machine execution engine. Production compilers like GCC or Clang are millions of lines long, making it difficult for students to inspect intermediate phase transformations. Sudarshan solves this problem by offering a transparent, zero-dependency 5-phase compiler pipeline in Python 3.

2. Objectives & Scope

Objectives: (1) Specify formal BNF syntax for imperative constructs; (2) Implement linear scanner with line/col tracking; (3) Construct LL(1) Recursive Descent Parser for AST generation; (4) Build lexically-scoped Symbol Table; (5) Linearize syntax trees into 3-Address Code quadruples; (6) Implement constant folding optimization pass; (7) Execute quadruples on a virtual machine model; (8) Centralize location-aware error handling.

Scope: Supports integer declarations, assignments, arithmetic/comparison operations, if/else, while loops, nested block scoping, and output statements. Non-goals include floating-point, strings, arrays, functions, and target assembly.

3. Compiler Design Concepts & Technology Stack

Technology Stack: Python 3.10+ Standard Library, Git/GitHub, VS Code. Zero external parsing libraries.

Phase 1: Lexical Analysis	â	â	â	â	"	Lexer, Tokenizer, Line/Col tracking (lexer.py)
Phase 2: Syntax Analysis	â	â	â	â	"	LL(1) Recursive Descent Parser & AST (parser.py)
Phase 3: Semantic Analysis	â	â	â	â	"	Tree-Walk Visitor & Scoped Symbol Table (semantic.py)
Phase 4: Intermediate Code	â	â	â	â	"	Three-Address Code Quadruples (tac.py)
Phase 4b: Code Optimization	â	â	â	â	"	Constant Folding & Dead Code Elimination (tac_opt.py)
Phase 5: Code Execution	â	â	â	â	"	Virtual Machine Interpreter (interpreter.py)

4. System Architecture & Methodology

Source Code (.mini)						
â						
â ...						
Lexer (lexer.py)	â	â	â	â	"	Token Stream (Line & Column tracking)
â						
â ...						
Parser (parser.py & ast_nodes.py)	â	â	â	â	"	Abstract Syntax Tree (AST)
â						
â ...						
Semantic Analyzer (semantic.py)	â	â	â	â	"	Lexically Scoped Symbol Table
â						
â ...						
TAC Generator (tac.py)	â	â	â	â	"	Three-Address Code (Quadruples)
â						
â ...						
TAC Optimizer (tac_opt.py)	â	â	â	â	"	Constant Folding Optimization

â
â ...
Interpreter (interpreter.py) â â â " Virtual Machine Execution & Output

5. Initial Working Prototype Validation

An initial working prototype has been fully implemented, tested, and validated.

Prototype Program (examples/valid.mini):

```
int sum = 0;
int i = 1;
while (i <= 5) {
    sum = sum + i;
    i = i + 1;
}
print(sum);
```

Prototype Execution Command:

```
./sudarshan examples/valid.mini --all
```

Verified Prototype Output: 15 (Status: Execution finished successfully)